VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT

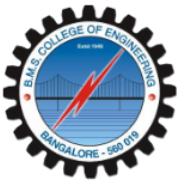
On

OPERATING SYSTEMS (23CS4PCOPS)

Submitted by

Nandini Aggarwal (1BF24CS192)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled **"OPERATING SYSTEMS"** carried out by Nandini Aggarwal (1BF24C192) , who is a bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Operating Systems Lab -**(23CS4PCOPS)**work prescribed for the said degree.

Sandhya A Kulkarni
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Course Outcomes:

CO1	Apply the different concepts and functionalities of Operating Systems.
CO2	Analyse various Operating system strategies and techniques
CO3	Demonstrate the different functionalities of Operating Systems.
CO4	Conduct practical experiments to implement the functionalities of Operating systems.

Lab Program 1:

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.
a) FCFS b) SJF c) Priority d) Round Robin

a) FCFS:

```
#include <stdio.h>
int main() {
    int n, i;
    int at[20], bt[20], ct[20], tat[20], wt[20], rt[20];
    float avg_wt = 0, avg_tat = 0, avg_rt = 0;
    printf("Enter number of processes: ");
    scanf("%d",&n);
    printf("Enter Arrival Time and Burst Time:\n");

    for(i=0;i<n;i++){
        printf("P%d AT: ",i+1);
        scanf("%d",&at[i]);
        printf("P%d BT: ",i+1);
        scanf("%d",&bt[i]);
    }
    ct[0] = at[0] + bt[0];
    for(i=1;i<n;i++){
        if(ct[i-1] < at[i])
            ct[i] = at[i] + bt[i];
        else
            ct[i] = ct[i-1] + bt[i];
    }
    for(i=0;i<n;i++){
```

```
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
        rt[i] = wt[i];
        avg_wt += wt[i];
        avg_tat += tat[i];
        avg_rt += rt[i];
    }
    printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\tRT\n");
    for(i=0;i<n;i++){
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, at[i], bt[i], ct[i], tat[i], wt[i], rt[i]);
    }
    printf("\nAverage Waiting Time = %.2f", avg_wt/n);
    printf("\nAverage Turnaround Time = %.2f", avg_tat/n);
    printf("\nAverage Response Time = %.2f\n", avg_rt/n);
    return 0;
}
```

Output:

```
Enter number of processes: 3
Enter Arrival Time and Burst Time:
P1 AT: 0
P1 BT: 5
P2 AT: 1
P2 BT: 3
P3 AT: 2
P3 BT: 5

Process AT      BT      CT      TAT     WT      RT
P1      0       5       5       5       0       0
P2      1       3       8       7       4       4
P3      2       5       13      11      6       6

Average Waiting Time = 3.33
Average Turnaround Time = 7.67
Average Response Time = 3.33
```

b) (i) SJF(Preemptive):

```
#include <stdio.h>
#include <limits.h>
int main(){
    int n,i,time=0,completed=0;
    int at[20], bt[20], rem_bt[20];
    int ct[20], tat[20], wt[20], rt[20];
    int first[20]={0};
    float avg_wt=0, avg_tat=0, avg_rt=0;
    printf("Enter number of processes: ");
    scanf("%d",&n);
    printf("Enter Arrival Time and Burst Time:\n");

    for(i=0;i<n;i++){
        printf("P%d AT: ",i+1);
        scanf("%d",&at[i]);
        printf("P%d BT: ",i+1);
        scanf("%d",&bt[i]);
        rem_bt[i]=bt[i];
        rt[i]=-1;
    }
    while(completed<n){
        int shortest=-1;
        int min_bt=INT_MAX;
        for(i=0;i<n;i++){
            if(at[i]<=time && rem_bt[i]>0 && rem_bt[i]<min_bt){
                min_bt=rem_bt[i];
```

```
            shortest=i;
        }
    }
    if(shortest==-1){
        time++;
        continue;
    }
    if(rt[shortest]==-1)
        rt[shortest]=time-at[shortest];
    rem_bt[shortest]--;
    time++;
    if(rem_bt[shortest]==0){
        completed++;
        ct[shortest]=time;
        tat[shortest]=ct[shortest]-at[shortest];
        wt[shortest]=tat[shortest]-bt[shortest];
        avg_wt+=wt[shortest];
        avg_tat+=tat[shortest];
        avg_rt+=rt[shortest];
    }
}
printf("\nProcess AT BT CT TAT WT RT\n");
for(i=0;i<n;i++){
    printf("P%d    %d  %d  %d  %d   %d  %d\n",
    i+1,at[i],bt[i],ct[i],tat[i],wt[i],rt[i]);
}
printf("\nAverage Waiting Time = %.2f",avg_wt/n);
printf("\nAverage Turnaround Time = %.2f",avg_tat/n);
```

```
    printf("\nAverage Response Time = %.2f\n",avg_rt/n);
    return 0;
}
```

Output:

```
Enter number of processes: 3
Enter Arrival Time and Burst Time:
P1 AT: 0
P1 BT: 5
P2 AT: 1
P2 BT: 3
P3 AT: 2
P3 BT: 4

Process AT BT CT TAT WT RT
P1     0  5  8  8   3  0
P2     1  3  4  3   0  0
P3     2  4  12  10   6  6

Average Waiting Time = 3.00
Average Turnaround Time = 7.00
Average Response Time = 2.00
```

(ii) SJF (Non-Preemptive):

```
#include <stdio.h>
#include <limits.h>
int main() {
    int n, i, time = 0, completed = 0;
    int at[20], bt[20], ct[20], tat[20], wt[20], rt[20];
    int done[20] = {0};
    float avg_wt = 0, avg_tat = 0, avg_rt = 0;
    printf("Enter number of processes: ");
    scanf("%d",&n);
    printf("Enter Arrival Time and Burst Time:\n");
```

```
for(i=0;i<n;i++){
    printf("P%d AT: ",i+1);
    scanf("%d",&at[i]);
    printf("P%d BT: ",i+1);
    scanf("%d",&bt[i]);
}

while(completed < n){
    int shortest = -1;
    int min_bt = INT_MAX;
    for(i=0;i<n;i++){
        if(at[i] <= time && done[i]==0 && bt[i] < min_bt){
            min_bt = bt[i];
            shortest = i;
        }
    }
    if(shortest == -1){
        time++;
        continue;
    }

    rt[shortest] = time - at[shortest];
    time += bt[shortest];
    ct[shortest] = time;
    tat[shortest] = ct[shortest] - at[shortest];
    wt[shortest] = tat[shortest] - bt[shortest];

    done[shortest] = 1;
```

```
            completed++;
            avg_wt += wt[shortest];
            avg_tat += tat[shortest];
            avg_rt += rt[shortest];
        }
        printf("\nProcess AT BT CT TAT WT RT\n");
        for(i=0;i<n;i++){
            printf("P%d    %d  %d  %d  %d   %d  %d\n",
            i+1,at[i],bt[i],ct[i],tat[i],wt[i],rt[i]);
        }
        printf("\nAverage Waiting Time = %.2f",avg_wt/n);
        printf("\nAverage Turnaround Time = %.2f",avg_tat/n);
        printf("\nAverage Response Time = %.2f\n",avg_rt/n);

        return 0;
    }
```

Output:

```
Enter number of processes: 3
Enter Arrival Time and Burst Time:
P1 AT: 0
P1 BT: 5
P2 AT: 1
P2 BT: 3
P3 AT: 2
P3 BT: 4

Process AT BT CT TAT WT RT
P1     0  5  5  5   0  0
P2     1  3  8  7   4  4
P3     2  4  12  10   6  6

Average Waiting Time = 3.33
Average Turnaround Time = 7.33
Average Response Time = 3.33
```

c) (i) Priority(Preemptive):

```
#include <stdio.h>
#include <limits.h>
int main()
{
   int n,i,time=0,completed=0;
   int at[20],bt[20],pr[20],rem_bt[20];
   int ct[20],tat[20],wt[20],rt[20];
   float avg_wt=0,avg_tat=0,avg_rt=0;
   printf("Enter number of processes: ");
   scanf("%d",&n);
   printf("Enter Arrival Time, Burst Time and Priority:\n");
   for(i=0;i<n;i++)
   {
      printf("P%d AT: ",i+1);
      scanf("%d",&at[i]);
      printf("P%d BT: ",i+1);
      scanf("%d",&bt[i]);
      printf("P%d Priority: ",i+1);
      scanf("%d",&pr[i]);
      rem_bt[i]=bt[i];
      rt[i]=-1;
   }
   while(completed<n)
   {
      int highest=-1;
      int min_pr=INT_MAX;
```

```
    for(i=0;i<n;i++)
    {
        if(at[i]<=time && rem_bt[i]>0 && pr[i]<min_pr)
        {
            min_pr=pr[i];
            highest=i;
        }
    }
    if(highest==-1)
    {
        time++;
        continue;
    }
    if(rt[highest]==-1)
        rt[highest] = time - at[highest];
    rem_bt[highest]--;
    time++;
    if(rem_bt[highest]==0)
    {
        completed++;
        ct[highest]=time;
        tat[highest]=ct[highest]-at[highest];
        wt[highest]=tat[highest]-bt[highest];
        avg_wt+=wt[highest];
        avg_tat+=tat[highest];
        avg_rt+=rt[highest];
    }
}
```

```
    printf("\nProcess AT BT PR CT TAT WT RT\n");
    for(i=0;i<n;i++)
    {
       printf("P%d    %d  %d  %d  %d  %d   %d  %d\n",
       i+1,at[i],bt[i],pr[i],ct[i],tat[i],wt[i],rt[i]);
    }
    printf("\nAverage Waiting Time = %.2f",avg_wt/n);
    printf("\nAverage Turnaround Time = %.2f",avg_tat/n);
    printf("\nAverage Response Time = %.2f\n",avg_rt/n);
    return 0;
}
```

Output:

```
Enter number of processes: 4
Enter Arrival Time, Burst Time and Priority:
P1 AT: 0
P1 BT: 5
P1 Priority: 2
P2 AT: 1
P2 BT: 3
P2 Priority:
1
P3 AT: 2
P3 BT: 4
P3 Priority: 3
P4 AT: 3
P4 BT: 2
P4 Priority: 2

Process AT BT PR CT TAT WT RT
P1     0  5  2  8  8   3  0
P2     1  3  1  4  3   0  0
P3     2  4  3  14  12   8  8
P4     3  2  2  10  7   5  5

Average Waiting Time = 4.00
Average Turnaround Time = 7.50
Average Response Time = 3.25
```

(ii)Priority (Non-preemptive):

```
#include <stdio.h>
#include <limits.h>
int main()
{
   int n,i,time=0,completed=0;
   int at[20],bt[20],pr[20];
   int ct[20],tat[20],wt[20],rt[20];
   int done[20]={0};
   float avg_wt=0,avg_tat=0,avg_rt=0;
   printf("Enter number of processes: ");
   scanf("%d",&n);
   printf("Enter Arrival Time, Burst Time and Priority:\n");
   for(i=0;i<n;i++)
   {
      printf("P%d AT: ",i+1);
      scanf("%d",&at[i]);
      printf("P%d BT: ",i+1);
      scanf("%d",&bt[i]);
      printf("P%d Priority: ",i+1);
      scanf("%d",&pr[i]);
   }
   while(completed<n)
   {
      int highest=-1;
      int min_pr=INT_MAX;
      for(i=0;i<n;i++)
```

```
    {
        if(at[i]<=time && done[i]==0 && pr[i]<min_pr)
        {
            min_pr=pr[i];
            highest=i;
        }
    }
    if(highest==-1)
    {
        time++;
        continue;
    }
    rt[highest] = time - at[highest];
    time += bt[highest];
    ct[highest] = time;
    tat[highest] = ct[highest] - at[highest];
    wt[highest] = tat[highest] - bt[highest];
    avg_wt += wt[highest];
    avg_tat += tat[highest];
    avg_rt += rt[highest];
    done[highest]=1;
    completed++;
}
printf("\nProcess AT BT PR CT TAT WT RT\n");
for(i=0;i<n;i++)
{
    printf("P%d    %d  %d  %d  %d  %d   %d  %d\n",
    i+1,at[i],bt[i],pr[i],ct[i],tat[i],wt[i],rt[i]);
```

```
    }
    printf("\nAverage Waiting Time = %.2f",avg_wt/n);
    printf("\nAverage Turnaround Time = %.2f",avg_tat/n);
    printf("\nAverage Response Time = %.2f\n",avg_rt/n);
    return 0;
}
```

Output:

```
Enter number of processes: 4
Enter Arrival Time, Burst Time and Priority:
P1 AT: 0
P1 BT: 5
P1 Priority: 2
P2 AT: 1
P2 BT: 3
P2 Priority: 1
P3 AT: 2
P3 BT: 4
P3 Priority: 3
P4 AT: 3
P4 BT: 2
P4 Priority: 2

Process AT BT PR CT TAT WT RT
P1     0  5  2  5  5   0  0
P2     1  3  1  8  7   4  4
P3     2  4  3  14  12   8  8
P4     3  2  2  10  7   5  5

Average Waiting Time = 4.25
Average Turnaround Time = 7.75
Average Response Time = 4.25
```

d) Round Robin:

```
#include <stdio.h>
int main() {
    int n, i, time = 0, remain;
    int at[20], bt[20], rem_bt[20];
    int ct[20], tat[20], wt[20], rt[20];
    int tq;
```

```
int gantt_p[100], gantt_t[100], g = 0;
float avg_wt = 0, avg_tat = 0, avg_rt = 0;

printf("Enter number of processes: ");
scanf("%d", &n);
printf("Enter Arrival Time and Burst Time:\n");

for(i = 0; i < n; i++) {
    printf("P%d AT: ", i+1);
    scanf("%d", &at[i]);

    printf("P%d BT: ", i+1);
    scanf("%d", &bt[i]);
    rem_bt[i] = bt[i];
    rt[i] = -1;
}
printf("Enter Time Quantum: ");
scanf("%d", &tq);
remain = n;
while(remain > 0) {
    int done = 1;
    for(i = 0; i < n; i++) {
        if(at[i] <= time && rem_bt[i] > 0) {
            done = 0;
            if(rt[i] == -1)
                rt[i] = time - at[i];
            gantt_p[g] = i;
            gantt_t[g] = time;
```

```
            g++;

            if(rem_bt[i] > tq) {
                time += tq;
                rem_bt[i] -= tq;
            } else {
                time += rem_bt[i];
                rem_bt[i] = 0;
                ct[i] = time;
                tat[i] = ct[i] - at[i];
                wt[i] = tat[i] - bt[i];
                avg_wt += wt[i];
                avg_tat += tat[i];
                avg_rt += rt[i];
                remain--;
            }
        }
    }
    if(done == 1)
        time++;
}
gantt_t[g] = time;
printf("\nGantt Chart:\n");
for(i = 0; i < g; i++) {
    printf("| P%d ", gantt_p[i]+1);
}
printf("|\n");
for(i = 0; i <= g; i++) {
```

```
        printf("%d   ", gantt_t[i]);
    }
    printf("\n\nProcess AT BT CT TAT WT RT\n");
    for(i = 0; i < n; i++) {
        printf("P%d    %d  %d  %d  %d   %d  %d\n",
        i+1, at[i], bt[i], ct[i], tat[i], wt[i], rt[i]);
    }
    printf("\nAverage Waiting Time = %.2f", avg_wt/n);
    printf("\nAverage Turnaround Time = %.2f", avg_tat/n);
    printf("\nAverage Response Time = %.2f\n", avg_rt/n);
    return 0;
}
```

Output:

```
Enter number of processes: 3
Enter Arrival Time and Burst Time:
P1 AT: 0
P1 BT: 5
P2 AT: 1
P2 BT: 3
P3 AT: 2
P3 BT: 4
Enter Time Quantum: 3

Gantt Chart:
| P1 | P2 | P3 | P1 | P3 |
0   3   6   9   11   12

Process AT BT CT TAT WT RT
P1     0  5  11  11   6  0
P2     1  3  6  5   2  2
P3     2  4  12  10   6  4

Average Waiting Time = 4.67
Average Turnaround Time = 8.67
Average Response Time = 2.00
```

Lab Program 2:

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

```
#include <stdio.h>

#define MAX 20

int main() {
    int n, i, time = 0, completed = 0;
    int at[MAX], bt[MAX], rt[MAX], ct[MAX], wt[MAX], tat[MAX];
    int type[MAX];
    printf("Enter number of processes: ");
    scanf("%d", &n);
    for(i = 0; i < n; i++) {
        printf("\nProcess %d\n", i);
        printf("Enter arrival time: ");
        scanf("%d", &at[i]);
        printf("Enter burst time: ");
        scanf("%d", &bt[i]);
        printf("Enter type (0 = System, 1 = User): ");
        scanf("%d", &type[i]);
        rt[i] = bt[i];
    }
    int done[MAX] = {0};
    while(completed < n) {
        int current = -1;
        for(i = 0; i < n; i++) {
            if(at[i] <= time && rt[i] > 0 && type[i] == 0) {
```

```
            current = i;
            break;
        }
    }
    if(current == -1) {
        for(i = 0; i < n; i++) {
            if(at[i] <= time && rt[i] > 0 && type[i] == 1) {
                current = i;
                break;
            }
        }
    }
    if(current == -1) {
        time++;
        continue;
    }
    rt[current]--;
    time++;
    if(rt[current] == 0) {
        ct[current] = time;
        tat[current] = ct[current] - at[current];
        wt[current] = tat[current] - bt[current];
        completed++;
    }
}
float avg_wt = 0, avg_tat = 0;
printf("\nID\tType\tAT\tBT\tCT\tWT\tTAT\n");
for(i = 0; i < n; i++) {
```

```
        printf("%d\t%s\t%d\t%d\t%d\t%d\t%d\n",i, (type[i] == 0) ? "System" "User",at[i], bt[i], ct[i], wt[i], tat[i]);
        avg_wt += wt[i];
        avg_tat += tat[i];
    }
    printf("\nAverage Waiting Time: %.2f", avg_wt/n);
    printf("\nAverage Turnaround Time: %.2f\n", avg_tat/n);
    return 0;
}
```

Output:

```
Enter number of processes: 04

Process 0
Enter arrival time: 0
Enter burst time: 5
Enter type (0 = System, 1 = User): 0

Process 1
Enter arrival time: 2
Enter burst time:
3
Enter type (0 = System, 1 = User): 1

Process 2
Enter arrival time: 1
Enter burst time: 4
Enter type (0 = System, 1 = User): 0

Process 3
Enter arrival time: 3
Enter burst time: 2
Enter type (0 = System, 1 = User): 1

ID      Type    AT      BT      CT      WT      TAT
0       System  0       5       5       0       5
1       User    2       3       12      7       10
2       System  1       4       9       4       8
3       User    3       2       14      9       11

Average Waiting Time: 5.00
Average Turnaround Time: 8.50
```

Lab Program 3:

Write a C program to simulate Real-Time CPU Scheduling algorithms:
a) Rate- Monotonic b) Earliest-deadline First c) Proportional scheduling

```
#include <stdio.h>
#include <math.h>
#define MAX 10
typedef struct {
    int id, bt, deadline, period;
    int ct, wt, tat;
} Process;
void sortEDF(Process p[], int n) {
    for(int i=0;i<n-1;i++) {
        for(int j=0;j<n-i-1;j++) {
            if(p[j].deadline > p[j+1].deadline) {
                Process temp = p[j];
                p[j] = p[j+1];
                p[j+1] = temp;
            }
        }
    }
}
void sortRMS(Process p[], int n) {
    for(int i=0;i<n-1;i++) {
        for(int j=0;j<n-i-1;j++) {
            if(p[j].period > p[j+1].period) {
                Process temp = p[j];
                p[j] = p[j+1];
```

```
                p[j+1] = temp;
            }
        }
    }
}
float calculateUtilization(Process p[], int n) {
    float u = 0;
    for(int i=0;i<n;i++) {
        u += (float)p[i].bt / p[i].period;
    }
    return u;
}
void calculateTimes(Process p[], int n) {
    int time = 0;
    for(int i=0;i<n;i++) {
        time += p[i].bt;
        p[i].ct = time;
        p[i].tat = p[i].ct;
        p[i].wt = p[i].tat - p[i].bt;
    }
}
void printTable(Process p[], int n, int isEDF) {
    printf("ID\tBF\t");
    if(isEDF) printf("Deadline\t");
    else printf("Period\t\t");
    printf("CT\tWT\tTAT\n");
    for(int i=0;i<n;i++) {
        printf("%d\t%d\t", p[i].id, p[i].bt);
```

```
        if(isEDF) printf("%d\t\t", p[i].deadline);
        else printf("%d\t\t", p[i].period);
        printf("%d\t%d\t%d\n", p[i].ct, p[i].wt, p[i].tat);
    }
}
void proportionalScheduling(Process p[], int n) {
    printf("\n===== Proportional Scheduling =====\n");
    float total = 0;
    for(int i=0;i<n;i++) total += p[i].bt;
    printf("ID\tBurst\tShare\n");
    for(int i=0;i<n;i++) {
        float share = p[i].bt / total;
        printf("%d\t%d\t%.2f\n", p[i].id, p[i].bt, share);
    }
}
int main() {
    int n;
    Process p[MAX], temp[MAX];
    printf("Enter number of processes: ");
    scanf("%d", &n);
    for(int i=0;i<n;i++) {
        p[i].id = i;
        printf("\nProcess %d:\n", i);
        printf("Burst Time: ");
        scanf("%d", &p[i].bt);
        printf("Deadline (EDF): ");
        scanf("%d", &p[i].deadline);
        printf("Period (RMS): ");
```

```
        scanf("%d", &p[i].period);
    }
    for(int i=0;i<n;i++) temp[i] = p[i];

    printf("\n===== Earliest Deadline First (EDF) =====\n");
    sortEDF(temp, n);
    calculateTimes(temp, n);

    float u = calculateUtilization(temp, n);
    printf("CPU Utilization: %.2f\n", u);
    if(u <= 1) printf("Schedulable (Utilization <= 1)\n");
    printTable(temp, n, 1);
    for(int i=0;i<n;i++) temp[i] = p[i];
    printf("\n===== Rate Monotonic Scheduling (RMS) =====\n");
    sortRMS(temp, n);
    calculateTimes(temp, n);
    float bound = n * (pow(2, (float)1/n) - 1);
    printf("CPU Utilization: %.2f\n", u);
    printf("RM Bound: %.4f\n", bound);
    if(u <= bound) printf("Schedulable (Utilization <= RM Bound)\n");
    printTable(temp, n, 0);
    proportionalScheduling(p, n);
    return 0;
}
```

Output:

```
Enter number of processes: 3

Process 0:
Burst Time: 1
Deadline (EDF): 4
Period (RMS): 5

Process 1:
Burst Time: 2
Deadline (EDF): 2
Period (RMS): 8

Process 2:
Burst Time: 1
Deadline (EDF): 6
Period (RMS): 10

===== Earliest Deadline First (EDF) =====
CPU Utilization: 0.55
Schedulable (Utilization <= 1)
ID      BF      Deadline        CT      WT      TAT
1       2       2               2       0       2
0       1       4               3       2       3
2       1       6               4       3       4

===== Rate Monotonic Scheduling (RMS) =====
CPU Utilization: 0.55
RM Bound: 0.7798
Schedulable (Utilization <= RM Bound)
ID      BF      Period          CT      WT      TAT
0       1       5               1       0       1
1       2       8               3       1       3
2       1       10              4       3       4

===== Proportional Scheduling =====
ID      Burst   Share
0       1       0.25
1       2       0.50
2       1       0.25
```

Lab Program 4:

Write a C program to simulate:
a) Producer-Consumer problem using semaphores b) Dining-Philosopher's problem

a) Producer-Consumer problem using semaphores

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0, out = 0;
sem_t empty, full, mutex;
void* producer(void* arg) {
   int item;
   for (int i = 0; i < 15; i++) {
      item = i;
      sem_wait(&empty);
      sem_wait(&mutex);
      buffer[in] = item;
      printf("Produced: %d at buffer[%d]\n", item, in);
      in = (in + 1) % BUFFER_SIZE;
      sem_post(&mutex);
      sem_post(&full);
   }
   return NULL;
}
```

```
void* consumer(void* arg) {
    int item;
    for (int i = 0; i < 15; i++) {
        sem_wait(&full);
        sem_wait(&mutex);
        item = buffer[out];
        printf("Consumed: %d from buffer[%d]\n", item, out);
        out = (out + 1) % BUFFER_SIZE;
        sem_post(&mutex);
        sem_post(&empty);
    }
    return NULL;
}
int main() {
    pthread_t p, c;
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    sem_init(&mutex, 0, 1);
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);
    pthread_join(p, NULL);
    pthread_join(c, NULL);
    sem_destroy(&empty);
    sem_destroy(&full);
    sem_destroy(&mutex);
    return 0;
}
```

Output:

```
Produced: 0 at buffer[0]
Produced: 1 at buffer[1]
Produced: 2 at buffer[2]
Produced: 3 at buffer[3]
Produced: 4 at buffer[4]
Consumed: 0 from buffer[0]
Consumed: 1 from buffer[1]
Consumed: 2 from buffer[2]
Consumed: 3 from buffer[3]
Consumed: 4 from buffer[4]
Produced: 5 at buffer[0]
Produced: 6 at buffer[1]
Produced: 7 at buffer[2]
Consumed: 5 from buffer[0]
Consumed: 6 from buffer[1]
Produced: 8 at buffer[3]
Produced: 9 at buffer[4]
Produced: 10 at buffer[0]
Produced: 11 at buffer[1]
Consumed: 7 from buffer[2]
Consumed: 8 from buffer[3]
Consumed: 9 from buffer[4]
Consumed: 10 from buffer[0]
Consumed: 11 from buffer[1]
```

b) Dining-Philosopher's problem

```
#include <stdio.h>

#include <pthread.h>

#include <semaphore.h>

#include <unistd.h>


#define N 5

sem_t forks[N];


void* philosopher(void* num) {
```

```
    int id = *(int*)num;
    while (1) {
        printf("Philosopher %d is thinking.\n", id);
        sleep(1);
        if (id == N - 1) {
            sem_wait(&forks[(id + 1) % N]);
            printf("Philosopher %d picked right fork %d.\n", id, (id + 1) % N);
            sem_wait(&forks[id]);
            printf("Philosopher %d picked left fork %d.\n", id, id);
        } else {
            sem_wait(&forks[id]);
            printf("Philosopher %d picked left fork %d.\n", id, id);
            sem_wait(&forks[(id + 1) % N]);
            printf("Philosopher %d picked right fork %d.\n", id, (id + 1) % N);
        }
        printf("Philosopher %d is eating.\n", id);
        sleep(1);
        sem_post(&forks[id]);
        sem_post(&forks[(id + 1) % N]);
        printf("Philosopher %d put down forks %d and %d.\n",
               id, id, (id + 1) % N);
    }
}
int main() {
    pthread_t ph[N];
    int ids[N];
    for (int i = 0; i < N; i++)
        sem_init(&forks[i], 0, 1);
```

```
    for (int i = 0; i < N; i++) {
        ids[i] = i;
        pthread_create(&ph[i], NULL, philosopher, &ids[i]);
    }
    for (int i = 0; i < N; i++)
        pthread_join(ph[i], NULL);
    return 0;
}
```

Output:

```
Philosopher 0 is thinking.
Philosopher 3 is thinking.
Philosopher 2 is thinking.
Philosopher 1 is thinking.
Philosopher 4 is thinking.
Philosopher 0 picked left fork 0.
Philosopher 0 picked right fork 1.
Philosopher 0 is eating.
Philosopher 3 picked left fork 3.
Philosopher 3 picked right fork 4.
Philosopher 3 is eating.
Philosopher 2 picked left fork 2.
Philosopher 3 put down forks 3 and 4.
Philosopher 4 picked right fork 0.
Philosopher 2 picked right fork 3.
Philosopher 2 is eating.
Philosopher 1 picked left fork 1.
Philosopher 3 is thinking.
Philosopher 0 put down forks 0 and 1.
Philosopher 0 is thinking.
Philosopher 4 picked left fork 4.
Philosopher 4 is eating.
Philosopher 2 put down forks 2 and 3.
Philosopher 2 is thinking.
```

Lab Program 5:

Write a C program to simulate:
a) Bankers' algorithm for the purpose of deadlock avoidance b) Deadlock Detection

a) Bankers' algorithm for the purpose of deadlock avoidance

```
#include<stdio.h>
int main() {
   int n,m,i,j,k,c=0,f[10]={0},a[10][10],max[10][10],need[10][10],av[10],s[10];

   printf("Enter number of processes: ");
   scanf("%d",&n);
   printf("Enter number of resources: ");
   scanf("%d",&m);

   printf("\nEnter Allocation Matrix:\n");
   for(i=0;i<n;i++)
      for(j=0;j<m;j++)
         scanf("%d",&a[i][j]);
   printf("\nEnter Maximum Demand Matrix:\n");
   for(i=0;i<n;i++)
      for(j=0;j<m;j++) {
         scanf("%d",&max[i][j]);
         need[i][j]=max[i][j]-a[i][j];
      }
   printf("\nEnter Available Resources:\n");
   for(i=0;i<m;i++)
      scanf("%d",&av[i]);
   while(c<n) {
```

```
        int found=0;
        for(i=0;i<n;i++) {
            if(!f[i]) {
                for(j=0;j<m;j++)
                    if(need[i][j]>av[j]) break;
                if(j==m) {
                    for(k=0;k<m;k++)
                        av[k]+=a[i][k];
                    s[c++]=i;
                    f[i]=1;
                    found=1;
                }
            }
        }
        if(!found) {
            printf("\nSystem is not safe");
            return 0;
        }
    }
    printf("\nSystem is in a safe state.\n");
    printf("Safe sequence is: ");
    for(i=0;i<n;i++) {
        printf("P%d",s[i]);
        if(i<n-1) printf(" -> ");
    }
}
```

Output:

```
Enter number of processes: 3
Enter number of resources: 3

Enter Allocation Matrix:
0 1 0
2 0 0
1 1 1

Enter Maximum Demand Matrix:
1 2 1
2 1 1
3 1 1

Enter Available Resources:
1 1 1

System is in a safe state.
Safe sequence is: P0 -> P1 -> P2
```

b) Deadlock Detection

```
#include<stdio.h>
    int main() {
        int n,m,i,j,k,f[10]={0},a[10][10],r[10][10],av[10],found;
        printf("Enter number of processes: ");
        scanf("%d",&n);
        printf("Enter number of resources: ");
        scanf("%d",&m);
        printf("\nEnter Allocation Matrix:\n");
        for(i=0;i<n;i++)
            for(j=0;j<m;j++)
                scanf("%d",&a[i][j]);
        printf("\nEnter Request Matrix:\n");]'
        for(i=0;i<n;i++)
            for(j=0;j<m;j++)
                scanf("%d",&r[i][j]);
```

```
    printf("\nEnter Available Resources:\n");
    for(i=0;i<m;i++)
        scanf("%d",&av[i]);
    do {
        found=0;
        for(i=0;i<n;i++) {
            if(!f[i]) {
                for(j=0;j<m;j++)
                    if(r[i][j]>av[j]) break;
                if(j==m) {
                    for(k=0;k<m;k++)
                        av[k]+=a[i][k];
                    f[i]=1;
                    found=1;
                }
            }
        }
    } while(found);
    int d=0;
    printf("\nProcesses in Deadlock:\n");
    for(i=0;i<n;i++)
        if(!f[i]) {
            printf("P%d ",i);
            d=1;
        }
    if(!d)
        printf("No deadlock detected. All processes can finish.");
}
```

Output:

```
Enter number of processes: 3
Enter number of resources: 3

Enter Allocation Matrix:
1 0 0
0 1 0
0 0 1

Enter Request Matrix:
0 1 0
0 0 1
1 0 0

Enter Available Resources:
0 0 0

Processes in Deadlock:
P0 P1 P2
```

Lab Program 6:

Write a C program to simulate the following contiguous memory allocation techniques.
a) Worst-fit b) Best-fit c) First-fit

```
#include <stdio.h>

void allocate(int b[], int m, int p[], int n, int type)
{
   int a[n];

   for (int i = 0; i < n; i++)
      a[i] = -1;

   for (int i = 0; i < n; i++)
   {
      int idx = -1;

      for (int j = 0; j < m; j++)
      {
         if (b[j] >= p[i])   // block can fit process
         {
            if (type == 0)   // First Fit
            {
               idx = j;
               break;
            }

            if (type == 1)   // Best Fit
            {
               if (idx == -1 || b[j] < b[idx])
                  idx = j;
            }

            if (type == 2)   // Worst Fit
            {
               if (idx == -1 || b[j] > b[idx])
                  idx = j;
            }
         }
      }

      if (idx != -1)
      {
```

```
            a[i] = idx;
            b[idx] = -1;   // only ONE process per block
        }
    }
    if (type == 0) printf("\n--- First Fit ---\n");
    if (type == 1) printf("\n--- Best Fit ---\n");
    if (type == 2) printf("\n--- Worst Fit ---\n");

    printf("Process No.\tProcess Size\tBlock No.\n");

    for (int i = 0; i < n; i++)
    {
        printf("%d\t\t%d\t\t", i + 1, p[i]);

        if (a[i] != -1)
            printf("%d\n", a[i] + 1);
        else
            printf("Not Allocated\n");
    }
}

int main()
{
    int m, n;

    printf("Enter number of memory blocks: ");
    scanf("%d", &m);

    int b[m], b1[m], b2[m], b3[m];

    printf("Enter sizes of %d memory blocks:\n", m);

    for (int i = 0; i < m; i++)
    {
        scanf("%d", &b[i]);
        b1[i] = b2[i] = b3[i] = b[i];
    }

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int p[n];

    printf("Enter sizes of %d processes:\n", n);
```

```
    for (int i = 0; i < n; i++)
        scanf("%d", &p[i]);

    allocate(b1, m, p, n, 0);
    allocate(b2, m, p, n, 1);
    allocate(b3, m, p, n, 2);

    return 0;
}
```

Output:

```
Enter number of memory blocks: 5
Enter sizes of 5 memory blocks: 100 500 200 300 600
Enter number of processes: 4
Enter sizes of 4 processes: 212 417 112 426
--- First Fit ---
Process No. Process Size    Block No.
1       212     2
2       417     5
3       112     3
4       426     Not Allocated

--- Best Fit ---
Process No. Process Size    Block No.
1       212     4
2       417     2
3       112     3
4       426     5

--- Worst Fit ---
Process No. Process Size    Block No.
1       212     5
2       417     2
3       112     4
4       426     Not Allocated
```

Lab Program 7:

Write a C program to simulate page replacement algorithms
a) FIFO b) LRU c) Optimal

```
#include <stdio.h>

int fifo(int p[], int n, int f) {
   int fr[10], i, j, k = 0, fault = 0, found;

   for(i = 0; i < f; i++) fr[i] = -1;

   for(i = 0; i < n; i++) {
      found = 0;
      for(j = 0; j < f; j++)
         if(fr[j] == p[i]) found = 1;

      if(!found) {
         fr[k] = p[i];
         k = (k + 1) % f;
         fault++;
      }
   }
   return fault;
}

int lru(int p[], int n, int f) {
   int fr[10], time[10], i, j, pos, min, fault = 0;

   for(i = 0; i < f; i++) {
      fr[i] = -1;
      time[i] = -1;
   }

   for(i = 0; i < n; i++) {
      int found = 0;
      for(j = 0; j < f; j++) {
         if(fr[j] == p[i]) {
            found = 1;
            time[j] = i;
         }
      }

      if(!found) {
```

```
            min = time[0];
            pos = 0;

            for(j = 1; j < f; j++) {
               if(time[j] < min) {
                  min = time[j];
                  pos = j;
               }
            }
            fr[pos] = p[i];
            time[pos] = i;
            fault++;
         }
      }
      return fault;
}

int optimal(int p[], int n, int f) {
   int fr[10], i, j, k, pos, farthest, fault = 0;

   for(i = 0; i < f; i++)
      fr[i] = -1;

   for(i = 0; i < n; i++) {
      int found = 0;

      for(j = 0; j < f; j++) {
         if(fr[j] == p[i]) {
            found = 1;
            break;
         }
      }

      if(!found) {
         pos = -1;
         farthest = -1;

         for(j = 0; j < f; j++) {
            int nextUse = -1;

            for(k = i + 1; k < n; k++) {
               if(fr[j] == p[k]) {
                  nextUse = k;
                  break;
               }
```

```
            }
            if(nextUse == -1) {
               pos = j;
               break;
            }
            if(nextUse > farthest) {
               farthest = nextUse;
               pos = j;
            }
         }
         fr[pos] = p[i];
         fault++;
      }
   }
   return fault;
}
int main() {
   int p[20], n, f, i;

   printf("Enter number of pages: ");
   scanf("%d", &n);

   printf("Enter pages: ");
   for(i = 0; i < n; i++)
      scanf("%d", &p[i]);

   printf("Enter number of frames: ");
   scanf("%d", &f);
   printf("\nFIFO Faults    = %d", fifo(p, n, f));
   printf("\nLRU Faults     = %d", lru(p, n, f));
   printf("\nOptimal Faults = %d", optimal(p, n, f));
}
```

Output:

```
Enter number of pages: 9
Enter pages: 1 2 1 3 1 2 4 5 1
Enter number of frames: 3

FIFO Faults    = 6
LRU Faults     = 6
Optimal Faults = 5
```